



THE OAK RIDGE ALGOL COMPILER
THE OAK RIDGE ALGOL COMPILER FOR THE
FOR THE
CONTROL DATA CORPORATION 1604
CONTROL DATA CORPORATION 1604
PRELIMINARY PROGRAMMER'S MANUAL

PRELIMINARY PROGRAMMER'S MANUAL

L. L. Bumgarner
L. L. Bumgarner

OAK RIDGE NATIONAL LABORATORY
OAK RIDGE NATIONAL LABORATORY



THE OAK RIDGE ALGOL COMPILER
THE OAK RIDGE ALGOL COMPILER FOR THE
FOR THE
CONTROL DATA CORPORATION 1604
CONTROL DATA CORPORATION 1604
PRELIMINARY PROGRAMMER'S MANUAL

PRELIMINARY PROGRAMMER'S MANUAL

L. L. Bumgarner
L. L. Bumgarner

OAK RIDGE NATIONAL LABORATORY
OAK RIDGE NATIONAL LABORATORY

pgdp.net

The Project Gutenberg eBook of The Oak Ridge ALGOL Compiler for the Control Data Corporation 1604

This eBook is for the use of anyone anywhere in the United States and most other parts of the world at no cost and with almost no restrictions whatsoever. You may copy it, give it away or re-use it under the terms of the Project Gutenberg License included with this eBook or online at www.gutenberg.org. If you are not located in the United States, you will have to check the laws of the country where you are located before using this eBook.

Title: The Oak Ridge ALGOL Compiler for the Control Data Corporation 1604

Author: L. L. Bumgarner

Release date: November 17, 2015 [eBook #50468]

Most recently updated: October 22, 2024

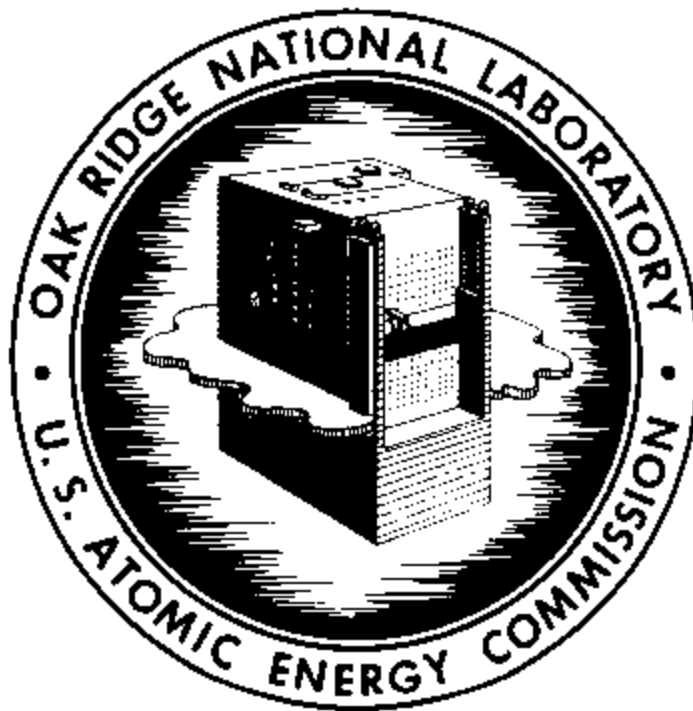
Language: English

Other information and formats: www.gutenberg.org/ebooks/50468

Credits: Produced by David Starner, Stephen Hutcheson, and the
Online Distributed Proofreading Team at
<http://www.pgdp.net>
(This book was produced from images made available by the
HathiTrust Digital Library.)

*** START OF THE PROJECT GUTENBERG EBOOK THE OAK
RIDGE ALGOL COMPILER FOR THE CONTROL DATA
CORPORATION 1604 ***

ORNL-3460
UC-32—Mathematics and Computers
TID-4500 (23rd ed.)



**THE OAK RIDGE ALGOL
COMPILER FOR THE CONTROL
DATA CORPORATION 1604
PRELIMINARY PROGRAMMER'S
MANUAL**

L. L. Bumgarner

OAK RIDGE NATIONAL LABORATORY

operated by

UNION CARBIDE CORPORATION

for the

U.S. ATOMIC ENERGY COMMISSION

Printed in USA. Price: \$1.25 Available from the
Office of Technical Services
U. S. Department of Commerce
Washington 25, D. C.

LEGAL NOTICE

This report was prepared as an account of Government sponsored work. Neither the United States, nor the Commission, nor any person acting on behalf of the Commission:

- A. Makes any warranty or representation, expressed or implied, with respect to the accuracy, completeness, or usefulness of the information contained in this report, or that the use of any information, apparatus, method, or process disclosed in this report may not infringe privately owned rights; or
- B. Assumes any liabilities with respect to the use of, or for damages resulting from the use of any information, apparatus, method, or process disclosed in this report.

As used in the above, "person acting on behalf of the Commission" includes any employee or contractor of the Commission, or employee of such contractor, to the extent that such employee or contractor of the Commission, or employee of such contractor prepares, disseminates, or provides access to, any information pursuant to his employment or contract with the Commission, or his employment with such contractor.

i

ORNL-3460

Contract No. W-7405-eng-26

Mathematics Division

THE OAK RIDGE ALGOL COMPILER FOR THE CONTROL DATA
CORPORATION
1604—PRELIMINARY PROGRAMMER'S MANUAL

L. L. Bumgarner

DATE ISSUED
JAN 30 1964

OAK RIDGE NATIONAL LABORATORY
Oak Ridge, Tennessee
operated by
UNION CARBIDE CORPORATION
for the
U.S. ATOMIC ENERGY COMMISSION

CONTENTS

I. <u>Introduction</u>	1
II. <u>Language Restrictions</u>	2
III. <u>Modes of Operation of the Compiler</u>	4
IV. <u>Input-Output and Intermediate Tape</u>	5
<u>Input-Output</u>	5
<u>READ</u>	5
<u>PAGE</u>	7
<u>Lists and the List Declaration</u>	7
<u>PRINT</u>	9
<u>WRITE</u>	9
<u>PUNCH</u>	10
<u>Formats and the Format Declaration</u>	10
<u>INPUT</u>	11
<u>OUTPUT</u>	12
<u>Intermediate Tape Procedures</u>	13
<u>BINREAD</u>	13
<u>BINWRITE</u>	14
<u>ENDFILE</u>	14
<u>REWIND</u>	14
<u>BACKUP</u>	14
<u>Tape-Checking Procedures</u>	14
<u>EOF</u>	15
<u>READERR</u>	15
<u>WRITERR</u>	15
V. <u>The External Declaration</u>	16
VI. <u>Standard Procedures</u>	16
VII. <u>Error Checking and Diagnostics</u>	17
VIII. <u>Running Programs</u>	19

<u>ALGOL Control System</u>	20
<u>EOP Card</u>	20
<u>Compile and Execute: ALGO</u>	21
<u>PROGRAM Card</u>	22
<u>Compile/Execute: ALDAP</u>	22
<u>ALDAP Control Statement</u>	22
<u>Job Deck: ALDAP Compilation/Execution</u>	23
<u>Examples</u>	25

APPENDICES

A. <u>Adjuncts to Algol 60</u>	30
B. <u>Hardware Representation</u>	32
C. <u>Structure of Procedure Calling Sequence</u>	35
D. <u>Internal Representation of Strings</u>	37
E. <u>Program Efficiency</u>	38
F. <u>Controversial Features of Algol 60</u>	40
G. <u>Fortran Subprograms in an Algol Program</u>	41

THE OAK RIDGE ALGOL COMPILER FOR THE CONTROL DATA CORPORATION 1604— PRELIMINARY PROGRAMMER'S MANUAL

L. L. Bumgarner

ABSTRACT

This document is a preliminary programmer's manual for use of the Control Data 1604 Algol Compiler. The compiler was constructed by the Programming Research Group of the Mathematics Division in cooperation with Control Data Corporation. A knowledge of Algol 60 is assumed. Included are descriptions of input-output facilities and details for operation under the monitor system.

I. Introduction

This document is to serve as a programmer's manual for the Algol compiler constructed as a cooperative project by Control Data Corporation and the Mathematics Division of Oak Ridge National Laboratory. The compiler is designed for the Control Data 1604 and 1604-A computers. The document is preliminary in that the compiler is not thoroughly tested and may undergo further development.

The reader is assumed to be familiar with Algol 60. The defining descriptions are the two reports on Algol 60 available in the following references:

1. P. Naur et al, "Report on the Algorithmic Language Algol 60," *Comm. Assoc. Comp. Mach.*, 3 (1960), No. 5, 299-314.
2. P. Naur et al, "Revised Report on the Algorithmic Language Algol 60," *Comm. Assoc. Comp. Mach.*, 6 (1963), No. 1, 1-17.

The second report clears up certain ambiguities that appeared in the first report. The reports are not easy reading for the novice. The following expositions are more readable:

1. Baumann, Bauer, Feliciano and Samelson, *Introduction to Algol*, 2 Prentice-Hall, Inc. (to be published in late 1963).
2. Bottenbruch, H., "Structure and Use of Algol 60," *Jour. Assoc. Comp. Mach.*, 9 (1962), No. 2, 161-221, and ORNL-3148.

The Baumann publication also contains the revised Algol 60 report.

Throughout this document various examples of statements and declarations appear without the semicolon which is always required for separating them. This is to avoid the implication that the semicolon is part of the statement or the declaration. In sentences, a comma or period may appear where a semicolon or other delimiter would be indicated in the context of a program.

Word delimiters rendered in bold-face type in the Algol report are herein indicated by underlining.

II. *Language Restrictions*

The compiler correctly handles programs written in Algol 60 subject to the following restrictions.

1. The use of an *integer label as an actual parameter* will cause an incorrect program to be compiled.
2. A GO TO statement with an *undefined switch designator* as the designational expression will cause incorrect operation of the final program.
3. *Type restrictions:*
 - (a) The exponentiation expression $x \uparrow y$ will have type **real** unless x is of type **integer** and y is a non-negative integer constant. This differs slightly from the definition in the Algol report but will generally cause no difficulty.

(b) In the construction

3

```
<if clause> <simple arithmetic expression>  
else <arithmetic expression>
```

the arithmetic expressions must have the same type, or else an incorrect program will be compiled. For example, in the statement

```
x := if a < b then z else w
```

z and w should both be declared **real** or both **integer**.

(c) In a procedure call (procedure statement or function call) each actual parameter having an arithmetic value must have the same type as the

corresponding formal parameter in the procedure declaration. The type of the formal parameter is that designated in the specification part if it appears there. If a formal parameter representing an arithmetic quantity does not appear in the specification part, it is assumed to be specified **real**. Full use of specifications is desirable for descriptive purposes and for optimization.

Caution. Restriction (c) is more likely to cause errors than the other restrictions. It is very easy to write $P(1,2)$ when the parameters of P are specified **real**, but incorrect coding will result. The call $P(1.0,2.0)$ works correctly.

4. *Standard procedure names* (see [section VI](#)) *used as parameters* in procedure calls will cause an incorrect program to be compiled. A call, therefore, such as

$P(\sin)$

is incorrect. Note, however, that a call of the type

$Q(\sin(x))$

causes no trouble. The case $P(\sin)$ can be programmed in another way. Make the declaration

4

```
real procedure sin 1 (t); real t;  
sin 1 := sin(t).
```

The call

$P(\sin\ 1)$

is then correct.

5. *Arrays called by value* are not handled. If an array identifier appears in the value part, an incorrect program will be compiled.

6. “*Dynamic*” *own arrays* are not handled. This means that all **own** arrays are treated as having constant subscript bounds; this constitutes one possible interpretation of the Algol 60 report. An **own** array may be

declared with variable subscript bounds, but only one allocation of storage will be made, and if the bounds change, this will be ignored.

7. *Recursive procedures* are not handled. This restriction encompasses all cases of a function designator appearing in the actual parameter part of a call of the same function, unless that function is a standard function. Thus $f(f(x))$ is not permitted in general, but $\sin(\sin(x))$ is allowed.

III. Modes of Operation of the Compiler

There are two distinct modes of operation: ALGO and ALDAP.

ALGO is a compile-and-execute mode in which the two phases cannot be separated. The Algol program is translated into a machine language program in core memory, and execution of the program immediately and automatically follows. There is no assembly program phase.

ALDAP makes use of the CODAP assembly program facilities. It is possible to compile procedures separately and reference them from an Algol program. The procedures may be written in Algol, CODAP or Fortran. This provision is made possible with the aid of the **external** declaration discussed in [section V](#). 5

The ALGO mode provides significantly faster compilation than the ALDAP mode for most programs. The target programs produced in the two modes are essentially the same. In the ALGO mode, program checkout may be done at the Algol language level. In the ALDAP mode, checkout may also be done at the machine and assembly language levels, and modifications may be made at these levels.

IV. Input-Output and Intermediate Tape

There are seven standard procedures for input-output, five for intermediate tape, and three for checking tape conditions. Two declarations, **format** and **list**, are additions to the language.

Input-Output

The input-output procedures are: READ, PAGE, PRINT, WRITE, PUNCH, INPUT, and OUTPUT.

READ

The READ procedure is used to input numbers and Boolean values. A READ statement has the form

```
READ (V1, V2, ..., Vn)
```

where n is any positive integer and each V_k is a variable. For example, the statement

```
READ (X, Y, A[1], B[1])
```

will input values into the four variables listed. For inputting values into an array, a statement such as the following might be used:

```
for I := 1 step 1 until 100 do READ (A[I]) .
```

The READ procedure inputs numbers and truth values. A number must be a legal Algol number (although an E may be substituted for the symbol ₁₀). For input into a Boolean variable, the truth values **true** and **false** are accepted; also, a non-negative number or a plus sign is interpreted as **false** and a negative number or a minus sign is interpreted as **true**. A blank is read as zero.

With the READ procedure, the type of a number on a data card does not have to be the same as the type of the variable to which it is assigned. Any necessary type conversions are done automatically. If N is the next number in the data, the statement

```
READ (V)
```

is equivalent to the statement

```
V := N .
```

The data cards are free field. The number of values per card, the length of numbers, and the number of spaces are arbitrary. A comma, however, must follow each number, *including the last one on the last data card*.

In reading a value into a subscripted variable, the current value of the subscript expression is not affected by that READ statement. For example, in the statement

```
READ (I, A[I])
```

the old value of I is used in A[I].

The READ procedure will input data from the standard input medium only. 7

PAGE

The PAGE procedure is used to cause a page ejection on the standard output medium. PAGE has no parameters. It is called by simply writing

Lists and the List Declaration

The input and output procedures described in the rest of this section, as well as the binary read and write procedures, make use of the concept of a list. A

^[1]
list is a sequence of expressions. An example is

U + V, C[0], if B then X else Y .

It may be inconvenient in some cases to write down all of the expressions explicitly. The *loop expression* ^[1] may be used as a shorthand device in a list. It is an Algol-like construction of which the following is an example:

for I := 1 step 1 until 1000 do A[I] .

This is equivalent to the list

A[1], A[2], ..., A[1000] .

The entity following **do** in a loop expression may itself be a list, but this list must be enclosed in parentheses if it contains more than one member.

The loop expression

for I := 1 step 1 until 1000 do (A[I], B[I])

is equivalent to the list

A[1], B[1], A[2], B[2], ..., A[1000], B[1000] .

The loop expression

**for I := 1 step 1 until 10 do (A[I], for J := 1
step 1 until 20 do B[I,J])**

is equivalent to the list

```
A[1], B[1,1], B[1,2], ..., B[1,20],
A[2], B[2,1], B[2,2], ..., B[2,20],
.....
A[10], B[10,1], B[10,2], ..., B[10,20] .
```

A list may be given a name through a list declaration. A list declaration has the form

```
list identifier := list .
```

Examples are:

```
list L := X, A + B
```

```
list M := for I := 1 step 1 until N do A[I] .
```

A list identifier may itself appear in a list. One of the above examples might be written with the aid of the following declaration:

```
list L := for J := 1 step 1 until 20 do B[I,J] .
```

The loop expression is then

```
for I:= 1 step 1 until 10 do (A[I], L) .
```

A list declaration obeys the same rules of syntax and scope as do other declarations.

A list identifier may be used as an actual parameter of a procedure call, with the requirement that the corresponding formal parameter be specified **list**. However, an actual list may appear as a parameter only in calls of the standard procedures, as described.

PRINT

The PRINT procedure is used to output numbers in a simple, rigid manner. A PRINT statement has the form

```
PRINT (list),
```

where list is described above. An example of a PRINT statement is

```
PRINT (A, if N = 0 then S else T).
```

A PRINT statement always puts out at least one line printer image. A line may contain up to 6 numbers, each of which is in scientific notation with 10 decimal places. Each number is right-justified in a field of 20 columns. (The format is 6E20.10.) The above PRINT statement will output two numbers in the first forty spaces, and the rest of the line will be blank. A PRINT statement such as

```
PRINT (for I := 1 step 1 until 10 do A[I])
```

will output one line of 6 numbers followed by one line of 4 numbers. Single spacing between lines is automatic.

The PRINT procedure always outputs on the standard output medium.

WRITE

The WRITE procedure is used to output strings. Examples of WRITE statements are:

```
WRITE ('TABLE')
```

```
WRITE (if D < 0 then 'TRUE' else 'FALSE') .
```

Each parameter must be a string expression (see [Appendix A](#) for definition of string expression). There may be any number of parameters, but each string will appear on a separate line. If a string is too long to go on one line, it will be continued on the next line. A string should not contain another string. Lines are single spaced. Each WRITE statement causes at least one line printer image to be put out.

10

The WRITE procedure always outputs on the standard output medium.

PUNCH

The PUNCH procedure is used to output numbers on punched cards in a form which can be input by the READ procedure. Each number punched will be followed by a comma. Each card punched may contain up to four numbers. Each number will be of type **real**, but since the READ procedure makes any necessary type conversions this is unimportant. A PUNCH statement has the same form as a PRINT statement. Each PUNCH statement causes at least one card image to be put out.

The PUNCH procedure always outputs on the standard punch medium.

Formats and the Format Declaration

The two input and output procedures remaining to be described make use of formats. The formats are exactly those used in Fortran, and readers unfamiliar with Fortran will find it necessary to refer to the Control Data Fortran-62 Reference Manual for details on the use of formats.

A format is treated as a string. Formats will be written, for example, as follows:

```
'(6E20.10)'
```

```
'(1H0, 9X, 5HTABLE, I3)' .
```

Note that the parentheses are part of the format, and *both parentheses and string quotes are required*.

As will be indicated below, a format string may appear explicitly in an INPUT or OUTPUT statement. If the same format string is used more than once, however, it may be convenient to give it a name through a format declaration. A format declaration has the form

11

```
format Identifier := '(Fortran format)' .
```

Examples are:

```
format F := '(6E20.10)'
```

```
format G := '(1H0, 9X, 5HTABLE, I3)' .
```

A format declaration obeys the same rules of syntax and scope as do other declarations.

Format identifiers may be used as parameters, and **format** is a specifier.

INPUT

The INPUT procedure is used to input numbers and Hollerith information in accordance with Fortran-type formats. An INPUT statement has one of the forms

```
INPUT (M,F,list)
```

```
INPUT (M,F)
```

where:

(1) M is the logical unit designation. M may be any arithmetic expression. If it is not integral-valued, the action

```
M := entier (M + 0.5)
```

will take place. The standard input unit is 50.

(2) F is a format expression. It may be an actual format string, a format identifier, a conditional format expression, or any variable which contains the starting address of a format string. *Caution.* In the case of a conditional format expression, format strings and format identifiers should not be mixed. For example, (a) and (b) below are permitted, but (c) will 12 cause an incorrect program to be compiled:

```
(a) if B then '(E20.7)' else '(E20.6)'
```

```
(b) if B then F1 else F2
```

```
(c) if B then F1 else '(E20.6)' .
```

(3) list is as defined previously. Of course, for INPUT all expressions must be variables.

The following are examples of an INPUT statement:

```
INPUT (50, '(4E20.8)', N, for I := 1 step 1 until N do A[I]).  
INPUT (if A < B then M else N, F, X, Y, Z) .
```

Each INPUT statement causes at least one card image to be read.

Note that the INPUT procedure does not make type checks between the data and the program variables. A floating point number, for example, is stored as such regardless of the type of the variable to which it is assigned.

Caution. It is strongly recommended that not both READ and INPUT be used in the same program. Each buffers ahead one card image. Furthermore, each INPUT statement causes at least one card image to be read while a READ statement may not cause a new card image to be read. Mixing the two statements will require quite careful use of blank cards in the data to allow for the buffering.

OUTPUT

The OUTPUT procedure is used to output numbers and Hollerith information in accordance with Fortran-type formats. An OUTPUT statement has one of the forms

```
OUTPUT (M, F)  
OUTPUT (M, F, list)
```

13

where M, F, and list are as indicated above. The following are examples of OUTPUT statements:

```
OUTPUT (51, '(5HTABLE)')  
OUTPUT (51, '(1H0,9X,10E10.2)', for I := 1 step 1 until 100  
      do A[I]) .
```

Each OUTPUT statement causes at least one line printer image to be put out. The standard output unit is 51, and the standard punch unit is 52.

Intermediate Tape Procedures

There are five standard procedures for making use of magnetic tape for auxiliary storage:

BINREAD, BINWRITE, ENDFILE, REWIND and BACKUP.

BINREAD

A BINREAD statement has the form

```
BINREAD (M, list)
```

where M and list are the same as for INPUT. Each BINREAD statement causes the designated unit to move forward one logical record, reading in binary format into the variables of the list. If fewer variables appear in the list than are on the record, only those values are read and the tape moves on to the end of the record. If more variables appear in the list than are on the record, this is treated as an error and the program is terminated.

The following is an example of a BINREAD statement:

```
BINREAD (6, for I := 1 step 1 until 1000 do A[I]) .
```

BINWRITE

A BINWRITE statement has the form

```
BINWRITE (M, list)
```

where *M* and *list* are the same as for *OUTPUT*. Each *BINWRITE* statement causes the values of the *list* expressions to be written in one logical record in binary format on the designated unit.

ENDFILE

An *ENDFILE* statement has the form

ENDFILE (*M*)

where *M* is a unit designation as before. The statement causes an end-of-file record to be written on the designated unit.

REWIND

A *REWIND* statement has the form

REWIND (*M*)

where *M* is a unit designation as before. The statement causes the designated unit to be rewound to the load point.

BACKUP

A *BACKUP* statement has the form

BACKUP (*M*)

where *M* is a unit designation as before. The statement causes the designated unit to be backspaced one logical record of binary information or one physical record of BCD information.

Tape-Checking Procedures

The checking procedures are: EOF, READERR, and WRITERR. These are Boolean procedures.

15

EOF

An EOF call has the form

EOF (M)

where M is a logical unit designation as before. It yields the value **true** if the previous read operation encountered an end-of-file or the previous write operation encountered an end-of-tape; otherwise it yields the value **false**.

An example of the use of an EOF call is:

if EOF(6) **then goto** ALARM .

READERR

A READERR call has the form

READERR (M)

where M is a logical unit designation as before. It yields the value **true** if the previous read operation produced a parity error; otherwise it yields the value **false**.

READERR should not be used for testing the operation of a READ statement. The READ procedure has its own facilities for checking, making multiple attempts in case of errors, and terminating the program if necessary.

WRITERR

A `WRITERR` call has the form

`WRITERR (M)`

where `M` is a logical unit designation as before. It yields the value **true** if the previous write operation produced a parity error; otherwise it yields the value **false**.

V. The External Declaration

An **external** declaration is required for each nonstandard library procedure or procedure compiled separately from the calling program, whether in Algol, Fortran or CODAP. Standard Algol procedures are described in [Section VI](#). Note that a CODAP subroutine must take account of the special structure of the Algol calling sequence as described in [Appendix C](#) or be treated as a Fortran subprogram. The use of Fortran subprograms is described in [Appendix G](#).

The *external* declaration has one of the following forms:

```
external I1, ..., In  
real external I1, ..., In  
integer external I1, ..., In  
Boolean external I1, ..., In
```

where each I_k is an identifier and n is any positive integer. A type declarator preceding the declarator **external** signifies a function procedure having that type. Note that no information about parameters appears in an **external** declaration. See [Appendix A](#) for syntactical definition.

In the ALGO mode, LIB cards must be included in the job deck for nonstandard library routines, in addition to the **external** declarations. Details are found in [Section VIII](#).

VI. Standard Procedures

Certain procedures are used without being declared. These include the standard functions listed in the Algol 60 report and the input-output and intermediate tape procedures. The complete list is as follows:

ABS
SIGN
SQRT
SIN
COS
ARCTAN
LN
EXP
ENTIER
EOF
READERR
WRITERR
FORTRANF
FTNF
READ
PAGE
PRINT
WRITE
PUNCH
INPUT
OUTPUT
BINREAD
BINWRITE
ENDFILE
REWIND
BACKUP
FORTRAN
FTN

These procedures are global to the program. They behave as though declared in a fictitious block surrounding the entire program.

VII. *Error Checking and Diagnostics*

In a complete compilation the compiler makes two passes on the Algol source program. If errors which the compiler cannot correct are detected in the first pass, then the second, or translation, pass will not be made. The following types of errors are detected:

1. syntactical error
2. undeclared identifier
3. identifier declared twice in the same block head
4. misspelled delimiter (corrected in many cases)
5. missing escape symbol (corrected unless both are missing for the same delimiter, in which case the delimiter is treated as an identifier).

The program listing and any diagnostics always appear on the standard output medium. In the case of a syntactical error, a message will appear in the program listing one or several lines below the error. The location of the error in the program will be further pinpointed in the line of symbols immediately below the error message. This line will be a short portion of the program with the last symbol in the line being the one which indicates the error. For example, a declaration might be out of place as follows:

```

      .
      .
      .
x := a + b; 'INTEGER' K;
**** LAST CHARACTER INDICATES SYNTACTICAL ERROR.
x := a + b; INTEGER
      .
```

·
·

In some cases the line below the message may differ slightly from the corresponding string of symbols above; for example, an identifier might be rendered by Ident. It is possible for a single syntactical error to cause more than one diagnostic.

A few syntactical errors are corrected by the compiler, and a message is put out to this effect. An example is a semicolon immediately preceding **else**.

According to the comment conventions of Algol, any string of symbols following **end** and not containing **end**, **else** or a semicolon is treated 19 as comment. As a result, the omission of one of these symbols following **end** does not always cause an error in compilation but will cause a portion of the program to be skipped over by the compiler. Thus for example, in

```
... x := a + b end for i := 1 step 1 ...
```

the FOR statement will be skipped at least in part. The compiler will put out a caution message in this and some other cases, but it will not change the program.

If an identifier is not declared (or possibly declared in the wrong place), a message is put out below the program listing together with the undeclared identifier.

The compiler does not check the type of identifiers. Therefore, such errors as a Boolean variable in an arithmetic expression, or the brackets of a subscripted variable replaced by parentheses, are not detected, and an incorrect program may be compiled.

VIII. *Running Programs*

The Algol program is punched on cards in the hardware representation described in [Appendix B](#). The format is essentially free field: spaces have no significance except within escape symbols and string quotes. *Only the first 72 columns, however, are interpreted by the compiler.* The remaining columns may be used for identification purposes. Care must be taken when a string is continued onto the next card, as *the continuation will begin in column 1*. The program listing will have the same format as the cards.

In the following discussion the symbol Ø signifies the letter O where necessary for emphasis, and the symbol Δ signifies a 7-9 punch in card column 1.

20

ALGOL Control System

The compiler operates under the ALGOL Control System. This system is a subordinate control routine of the Master Control System of the CO-OP Monitor Programming System. ALGOL is quite similar to the subordinate control routine COOP.

ALGOL is called with an MCS (Master Control System) card having ALGOL punched beginning in column 2. Other details of this card are available in descriptions of the CO-OP Monitor. It should be noted in selecting a standard recovery procedure that the concept of COMMON is not used in Algol.

Following the MCS card will be a control card giving instructions to the control routine ALGOL. It will name one of the following routines: ALGO, ALDAP, EXECUTE, BINARY, FORTRAN, REWIND or DEFINE. These will be discussed below.

EOP Card

The EOP (end-of-program) card has the characters 'EØP' punched in columns 10-14.

In the ALGO mode, one EOP card must be used to terminate the program.

In the ALDAP mode, one EOP card must be used to terminate each Algol program or Algol procedure being compiled separately.

21

Compile and Execute: ALGO

The ALGO mode of running an Algol program is the simplest and the fastest. It will be the more suitable for a large number of programs. Unless the programmer has special reasons for using the ALDAP mode, the ALGO mode is recommended.

The Algol program must be self-contained except for standard procedures and library procedures on the library-systems tape. The job deck must have the following cards in the specified order:

1. MCS control card.

The subordinate control routine name must be ALGØL.

2. ALGOL control card.

This will appear as

Δ ALGØ. or Δ ALGØ, t. where t is an integer specifying a time limit in minutes for compilation and execution.

(The period is required on every control card.)

3. LIB cards.

If necessary. One LIB card is required for each non-standard library procedure called in the program, namely those declared **external**. The format of a LIB card is as follows: the characters LIB punched in columns 10-12 and the name of a library entry point beginning in column 20. There may be no more than 20 LIB cards.

4. PROGRAM card.

If desired. This may be used to identify the program. Its format is described in the next paragraph.

5. Algol program deck.

6. EOP card.

7. Data.

If required.

PROGRAM Card

The PROGRAM card is optional. It is useful for identification purposes, and in the ALDAP mode it serves to name the program entry point.

The format of the card is free field. The characters PRØGRAM must appear followed by the program name, which must be alphanumeric.

Compile/Execute: ALDAP

The ALDAP mode is used to compile an Algol program or procedure to a relocatable binary or a CODAP format. Execution is optional. For compilation only, the program deck may consist of any mixture of Algol programs and procedures, any number of which may be in CODAP. If execution is desired, part or all of the program deck may have been previously compiled, so that the deck may have Algol, CODAP and relocatable binary cards.

ALDAP Control Statement

The format of the ALDAP statement is:

$$\Delta\text{ALDAP}, L, B, n.$$

where

L is a program listing key,
 B is a punched card output key,
 n is a logical unit number.

A period may terminate the statement at any point, with remaining fields treated as zero.

If the program listing key (L) is a 1, an assembled listing of the CODAP object code will be produced on the standard output medium. If the key is zero or blank, no such listing will be produced. A listing of the Algol 23 program and any diagnostics will always be produced on the standard output medium.

If the punched card output key (B) is a 1, a relocatable binary deck will be produced on the standard punch medium. If the key is a 2, a CODAP symbolic deck will be produced on the standard punch medium. If the key is a 3, both a symbolic deck and a relocatable binary deck will be produced on the standard punch medium, with the symbolic deck appearing first. If the key is zero or blank, no deck will be produced.

The logical unit number (n) specifies the unit which is to be the load-and-go tape if it is one of the integers 1-49 or 56. If n is some other integer or blank, no load-and-go tape will be written. The load-and-go tape is required when execution of the program is to follow.

Examples:

(a) $\Delta\text{ALDAP}, 1, 1, 56.$

This statement will cause the Algol/CODAP deck to be compiled, an assembled listing to be produced on the standard output medium, a relocatable binary deck to be produced on the standard punch medium, and a load-and-go tape written on logical unit 56.

(b) Δ ALDAP, 1.

This statement will cause the Algol/CODAP deck to be compiled, and an assembled listing to be produced on the standard output medium.

Job Deck: ALDAP Compilation/Execution

For *compilation only* of an Algol/CODAP program deck, the job deck should contain the following cards in the specified order:

1. MCS control card. 24
With ALGØL as the subordinate control routine name.
2. ALGOL control card.
With the appropriate ALDAP control statement.
3. PROGRAM card.
If desired.
4. Program deck.
Any mixture of Algol and CODAP programs and procedures, with all their subroutines except the standard procedures and those on the library-systems tape. *Each Algol program or procedure must be terminated by an EOP card.*
5. FINIS card.
This card contains the characters FINIS punched in columns 10-14. It signals the end of all compilations.

For *compilation and execution* of an Algol/CODAP program deck, a load-and-go tape must be requested in the ALDAP control statement. If no relocatable binary cards follow the last subprogram to be compiled, then the program deck must be terminated by an EOP card which is *in addition to* the EOP card or END card (the latter for a CODAP subprogram) which terminates the last program or procedure. The FINIS card then follows this additional EOP card. An EOP card always causes a TRA card image to be written on the load-and-go tape.

The control statements EXECUTE, BINARY, FORTRAN, REWIND and DEFINE may be used as described in the “CO-OP Monitor Programmer’s

Guide”. BINARY is useful for loading a relocatable binary deck onto the load-and-go tape prior to compilation of an Algol calling program, where the subprogram in relocatable form might have the same name as a library routine. If the Algol program preceded the relocatable deck, the library routine would be fetched by the loader and an error indication given.

The CO-OP control statements LOAD and EXECUTER are not used by ALGOL. 25

Examples

Each of the following examples describes a job deck which illustrates a different way of compiling and executing the same Algol program. The program calls a library procedure with entry point named BESSEL, and the program contains at least one other procedure. On the MCS card only the first field is indicated, as the others may vary from one installation to another.

Example 1

This job uses the ALGØ mode.

```
ΔALGØL, . . . .
```

```
ΔALGØ.
```

```
LIB BESSEL
```

```
PRØGRAM SAMPLE
```

```
Algol Program (with external declaration of BESSEL)
```

```
'EØP'
```

```
Data
```

Example 2

This job uses the ALDAP mode, compiling the entire program at once. The ALDAP control statement calls for an assembled listing, a binary deck, and a load-and-go tape on logical unit 56. The execute card gives a two minute time limit on the execution.

```
ΔALGØL, . . . .
```

26

```
ΔALDAP, 1, 1, 56.
```

```
PRØGRAM SAMPLE
```

```
Algol Program (with external declaration of BESSEL)
```

```
'EØP'
```

```
'EØP'
```

```
FINIS
```

```
ΔEXECUTE, 2.
```

```
Data
```

Example 3

This job consists simply of the execution of the relocatable program deck obtained in [example 2](#).

```
ΔALGØL, . . . .
```

```
ΔEXECUTE, 2.
```

```
Relocatable Deck
```

```
Data
```

Example 4

This example is similar to [example 2](#). Here the main program and one of its procedures are to be compiled separately.

```
ΔALGØL, . . . .
```

27

```
ΔALDAP, 1, 1, 56.
```

PRØGRAM SAMPLE

Algol Program (with **external** declaration of both BESSEL
and the procedure being compiled separately)

'EØP'

Algol Procedure

'EØP'

'EØP'

FINIS

ΔEXECUTE, 2.

Data

Example 5

In this example the procedure which was compiled separately in [example 4](#) is being compiled by itself, i.e., the calling program is not in the deck at all. Of course there is no execution in this case. Note that no load-and-go tape is requested and only one EOP card is used. There cannot be a PROGRAM card.

ΔALGØL,

ΔALDAP, 1, 1.

Algol Procedure

'EØP'

FINIS

Example 6

Here the procedure compiled by itself in [example 5](#) appears in the program deck in relocatable binary form, while the calling program is in the Algol language.

ΔALGØL,

ΔALDAP, 1, 1, 56.

PRØGRAM SAMPLE

Algol Program (with **external** declaration of both BESSEL
and the procedure in relocatable form)

'EØP'

FINIS

ΔEXECUTE, 2.

Relocatable Deck

Data

The relocatable deck here must be terminated by two TRA cards. One of these is generated by the compiler when it processes the EOP card which must terminate the procedure for compilation, as in [example 5](#). The second TRA card can be obtained by using a second EOP card, as in [example 2](#). Alternatively, the second TRA card can be added to the relocatable deck before execution. Note that this second TRA card must not be used when the relocatable deck is loaded by a BINARY control statement. This is illustrated in the next example.

Example 7

In this case the previously compiled procedure has the same name as a routine on the library-systems tape.

ΔALGØL,

ΔBINARY, 56.

Relocatable Deck (terminated by one TRA card)

ΔALDAP, 1, 1, 56.

PRØGRAM SAMPLE

Algol Program (with **external** declaration of both BESSEL
and the procedure in relocatable form)

'EØP'

'EØP'

FINIS

ΔEXECUTE, 2.

Data

The logical unit number on the BINARY control statement must agree with that which specifies the load-and-go tape in the ALDAP control statement.

APPENDIX A

Adjuncts to Algol 60

List Entities

The delimiter *list* is a declarator and a specifier.

```
<list identifier> ::= <identifier>

<loop expression> ::= <for clause> <arithmetic expression>
    | <for clause> <loop expression>
    | <for clause> (<list>)

<list element> ::= <arithmetic expression>
    | <loop expression>
    | <list identifier>

<list> ::= <list element>
    | <list>, <list element>

<list declaration> ::= list <list identifier> := <list>
```

Format Entities

The delimiter **format** is a declarator and a specifier.

```
<format identifier> ::= <identifier>

<simple format expression> ::= '(<Fortran format> [2])'
    | <format identifier>
```

```
<format expression> ::= <simple format expression>  
    | <if clause> <simple format expression>  
      else <format expression>  
  
<format declaration> ::= format <format identifier> :=  
    <format expression>
```

31

String Expression

```
<string expression> ::= <string>  
    | <if clause> <string>  
      else <string expression>
```

External Declaration

The delimiter **external** is a declarator.

```
<external identifier> ::= <identifier>  
  
<external list> ::= <external identifier>  
    | <external identifier>, <external list>  
  
<external declaration> ::= external <external list>  
    | <type> external <external list>
```

32

APPENDIX B

Hardware Representation

One keypunch character is reserved as an “escape symbol”, which we shall here suppose is the apostrophe. This symbol is used to delineate word delimiters and truth values, which are written in boldface type in Algol reference language and publication language and indicated by underlining in this manual. The hardware representation of a word delimiter such as **begin** is therefore 'BEGIN'. No distinction is made between upper and lower case letters in the hardware language.

The transliteration rules for the non-word delimiters are comprised in the following table. This assumes a 48 character hardware set and is consistent with the usage in the ALCOR group. For some basic symbols alternatives are tolerated, as indicated.

<i>Reference Hardware</i>	<i>Tolerated Hardware</i>
<	'LS' 'LESS'
≤	'LQ' 'LSEQ', 'NOTGREATER', 'NOT GREATER'
=	'EQ' 'EQUAL'
≥	'GQ' 'GREQ', 'NOTLESS', 'NOT LESS'
>	'GR' 'GREATER'
≠	'NQ' 'NTEQ', 'NOTEQUAL', 'NOT EQUAL'
¬	'NOT'
∧	'AND'
∨	'OR'
⊃	'IMP' 'IMPLIES', 'IMPL'
≡	'EQV' 'EQUIV'
1 0	' ' 'E', 'T'

×	*	
↑	**	'POWER'
÷	//	'DIV'
:	..	
;	\$	.,
:=	=	.,, ..=
[	(/	
]	/)	
,	"	'('
,	"	')'

In the case of the string quotes, the tolerated symbols are required for the inner strings of a nest of strings.

33

Actually, the compiler can tolerate many other spellings of word delimiters because of its facility for correcting misspellings.

The delimiter **go to** is accepted with or without the space between the two words, but it is treated as a single delimiter: 'GOTO' or 'GO TO'.

The compiler can also accept a 64 character hardware representation: the full set available on the line printer. In preparing programs, overpunching is used on the 48 character keypunch in this case. The table below indicates the keypunching rules in use at Oak Ridge National Laboratory.

34

Reference Hardware

<	1-8 punch
≤	1-5 punch
≥	1-9 punch
>	2-7 punch
≠	2-6 punch
Λ	3-7 punch
∨	2-4 punch
1 0	1-6 punch
↑	2-5 punch
÷	3-5 punch
:	2-8 punch

;	2-9	punch
[	3-6	punch
]	3-4	punch

The other basic symbols are either in the 48 character set or are replaced by word delimiters as above. The symbol := is treated as two symbols in the 64 character set, and = is punched as such.

APPENDIX C

Structure of Procedure Calling Sequence

The following information is necessary for the user writing a non-Algol procedure to be called from an Algol program. The calling sequence differs from that found in many other languages.

The first word of the non-Algol procedure must have a simple jump instruction in its upper half, and the exit line is provided by a jump to this first word. The entry automatically causes the proper return address to be placed in the address portion of the first half-word.

Upon entry to the procedure, index register six contains an address which is used to reference each parameter. To establish linkage with the first parameter, the instruction

```
LDA 6 0
```

is performed. This brings into the accumulator a word of one of the following types:

1. SLJ 0 ENA V
2. SLJ 0 RTJ L

In case (1), *v* is the address of the parameter. In case (2), *L* is the starting address of a piece of coding for computing the address of the parameter and leaving it in the accumulator (if the parameter is an expression, the address in the accumulator will be that of a temporary containing its value). Case (1) always holds if the parameter is a simple variable, string, array identifier, switch identifier, or procedure identifier. In case (2) the same temporary will be used for all the expressions.

Both cases can be provided for by setting aside two locations for each parameter in the procedure body and placing the instruction

36

```
SLJ *-1
```

in the upper half of each second location. Then after

```
LDA 6 0
```

mentioned above,

```
STA RES1,
```

where RES1 is the first reserved location for the first parameter, makes the two locations into a closed subroutine. After this, the instruction

```
RTJ RES1
```

causes the address of the first parameter to be placed in the accumulator anytime it is performed. This accommodates expressions called by name.

In general, the K^{th} parameter is referenced as above, but beginning with

```
LDA 6 (K - 1).
```

This description does not apply to the standard procedures, each of which has its own special calling sequence.

37

APPENDIX D

Internal Representation of Strings

The address representing a string is that of the first word of string characters. Each left string quote is represented internally by the word

00 ... 03454 ,

and each right string quote by

00 ... 05474 .

The characters of the string which are not string quotes are packed in BCD eight characters per word. These words are in the natural order, the first immediately following the left string quote and the last immediately followed by the right string quote. If the last word before a right quote is not full, the rest of that word is filled out with zeros (not BCD blanks).

APPENDIX E

Program Efficiency

The following information may be of interest to programmers desiring an efficient program:

1. The FOR statement is defined with more generality than is useful in most programs. In particular, the arithmetic expressions in the FOR clause are allowed to change in value during execution of the FOR statement. The compiler does not attempt to determine which FOR statements make use of this flexibility and treats all of them in the most general way. Therefore, in a statement such as

```
for I := 1 step M + N until abs(A - B) do ... ,
```

the expression $M + N$ is evaluated twice for each iteration, and the expression $\text{abs}(A - B)$ is evaluated once for each iteration. If M , N , A , and B do not change in the loop, this is unnecessary. Such inefficiency can be avoided by programming in a slightly different way. The above example can be written as follows:

```
T1 := M + N; T2 := abs(A - B) ;  
for I := 1 step T1 until T2 do ... .
```

2. The concept of call by value is a device applied to procedures to eliminate unneeded flexibility in procedure calls. If a parameter having a value is referenced more than once in the procedure body and the flexibility of call by name is not needed, then the program is more efficient if the parameter is included in the value part of the procedure

heading. If such a parameter is referenced only once, it is more efficient if it is not included in the value part.

3. Array identifiers which are parameters should be specified.

APPENDIX F

Controversial Features of Algol 60

A few features of the language have been subject to more than one interpretation. Fortunately, the vast majority of programs will not involve these ambiguities, but for the few that do it will be necessary to know what decisions the compiler makes. This appendix indicates these decisions for the more controversial areas.

1. Side effects in function designators. The evaluation of primaries in expressions is not strictly left to right allowing for precedence rules. In particular, the value of a variable in an expression is never stored in a temporary simply to preserve its value from change by the evaluation of a function designator in the expression. Otherwise, the evaluation does proceed from left to right and according to precedence rules, including the referencing of formal parameters and the calculation of the address of subscripted variables. All function designators are evaluated in Boolean expressions.

2. *Own* variables and arrays in procedures. The **own** quantities local to the body of a procedure which is called from more than one point in a program record the history of the procedure as opposed to a history of each point of reference. In other words, only one copy of the **own** quantities is preserved.

APPENDIX G

Fortran Subprograms in an Algol Program

The standard procedures FORTRAN, FORTRANF, FTN, and FTNF are used to call compiled Fortran subroutines and functions from within an Algol program. Each procedure has one parameter which is a call of the desired Fortran subprogram. The Fortran subprogram must be declared **external** as described in [Section V](#).

The use of these procedures simply causes a Fortran calling sequence to be generated by the compiler. Of course the subprogram could be written in CODAP as well as Fortran, provided it is designed to link through a Fortran-type calling sequence.

The procedures are used as follows:

FORTRAN—generates a Fortran 62 calling sequence for a subroutine

FORTRANF—generates a Fortran 62 calling sequence for a function

FTN—generates a Fortran 63 calling sequence for a subroutine

FTNF—generates a Fortran 63 calling sequence for a function

Each of these procedures is standard, i.e., available without declaration. FORTRANF and FTNF are used in expressions.

Examples:

```
x := FTNF (ALPHA(T,A[0,0]))
```

```
FORTRAN (SUB(I + J)) .
```

The following restrictions must be observed: labels, procedures with no parameters, standard procedure names, and array names cannot be used as arguments of a call of a Fortran subprogram. However, in the case of an array, the subscripted variable which is the first element of the array 42 will satisfy a Fortran subroutine which has an array name as a formal parameter. The name of the Fortran subprogram cannot be a formal parameter. Literals must be enclosed in string quotes.

Footnotes

[1] See [Appendix A](#) for syntactical definition.

[2] For definition of Fortran format, see Control Data Fortran-62 Reference Manual.

43

Acknowledgment

The author was greatly assisted in the preparation of this document by several persons who have contributed labors or advice to the construction of the compiler. These include N. B. Alexander and A. A. Grau, also K. A. Wolf of Control Data Corporation, and especially R. G. Stueland of Control Data Corporation.

45

ORNL-3460
UC-32—Mathematics and Computers
TID-4500 (23rd ed.)

INTERNAL DISTRIBUTION

1. Biology Library
- 2-4. Central Research Library
5. Reactor Division Library
- 6-7. ORNL--Y-12 Technical Library Document Reference Section

8-27. Laboratory Records Department

28. Laboratory Records, ORNL R.C.

29. R. K. Adams

30. Nancy Alexander

31. E. D. Arnold

32. Don Arnurius

33. George J. Atta

34. Susie E. Atta

35. S. J. Ball

36. J. E. Bigelow

37. R. E. Biggers

38. Craig Brandon

39. J. C. Bresee

40-41. L. L. Bumgarner

42. W. R. Burrus

43. H. P. Carter

44. D. K. Cavin

45. Arline Culkowski

46. W. Davis, Jr.

47. H. J. de Bruin

48. P. B. DeNee

49. A. C. Downing

50. L. C. Emerson

51. Margaret Emmett

52. R. L. Ferguson

53. B. R. Fish

54. P. A. Haas

55. M. Feliciano

56. Barbara Ann Flores

57. T. B. Fowler

58. R. E. Funderlic (K-25)

59. D. A. Gardiner

60. C. D. Griffies

61. D. A. Griffin

62. D. G. Gosslee

63. M. T. Harkrider

64. M. C. Hill

65. A. S. Householder
66. W. H. Jordan
67. H. W. Joy
68. F. B. K. Kam
69. George Kidd
70. L. J. King
71. Ann Klein
72. K. A. Kraus
73. C. E. Larson
74. M. E. LaVerne
75. Elmon Leach
76. R. P. Leinius
- 77-78. M. P. Lietzke
79. Ernie McDaniel
80. C. D. Martin
81. K. O. Martin
82. Betty F. Maskewitz
83. R. P. Milford
84. F. L. Miller, Jr.
85. R. V. Miskell
86. S. E. Moore
87. J. F. Murdock
88. C. W. Nestor, Jr.
89. V. K. Pare
90. Carl E. Parker
91. S. K. Penny
92. A. M. Perry
93. D. C. Ramsey
94. M. T. Robinson
95. R. M. Rush
96. Y. Shima
97. J. E. Simpkins
98. M. J. Skinner
99. C. D. Scott
100. C. D. Susano
101. J. A. Swartout
102. M. E. Tsagaris

- 103. D. K. Trubey
- 104. J. S. Watson
- 105. A. M. Weinberg
- 106. M. E. Whatley
- 107. C. S. Williams
- 108. H. A. Wright
- 109. Y-12 Central Files
- 110. J. H. Zeigler (K-25)
- 111. H. Zeldes

EXTERNAL DISTRIBUTION

- 112. T. H. Elrod, Control Data Corporation, Computer Division, 3330 Hillview Avenue, Palo Alto, California
- 113. A. A. Grau, Department of Mathematics, Northwestern University, Evanston, Illinois
- 114. R. G. Stueland, Control Data Corporation, Computer Division, 3330 Hillview Avenue, Palo Alto, California
- 115. K. A. Wolf, Control Data Corporation, Programing Systems, 501 Park Avenue, Minneapolis 15, Minnesota
- 116. R. A. Zemlin, Control Data Corporation, Computer Division, 3330 Hillview Avenue, Palo Alto, California
- 117. Research and Development Division, AEC, ORO
- 118-728. Given distribution as shown in TID-4500 (23rd ed.) under Mathematics and Computers category (75 copies--OTS)

Transcriber's Notes

In the HTML version, represented code in a monospaced font, with keywords in **boldface** as in the *Revised Report*.

Corrected a few palpable typos.

Created an original cover image, using elements from the printed book, for free and unrestricted use with this eBook.

*** END OF THE PROJECT GUTENBERG EBOOK THE OAK RIDGE
ALGOL COMPILER FOR THE CONTROL DATA CORPORATION 1604

Updated editions will replace the previous one—the old editions will be renamed.

Creating the works from print editions not protected by U.S. copyright law means that no one owns a United States copyright in these works, so the Foundation (and you!) can copy and distribute it in the United States without permission and without paying copyright royalties. Special rules, set forth in the General Terms of Use part of this license, apply to copying and distributing Project Gutenberg™ electronic works to protect the PROJECT GUTENBERG™ concept and trademark. Project Gutenberg is a registered trademark, and may not be used if you charge for an eBook, except by following the terms of the trademark license, including paying royalties for use of the Project Gutenberg trademark. If you do not charge anything for copies of this eBook, complying with the trademark license is very easy. You may use this eBook for nearly any purpose such as creation of derivative works, reports, performances and research. Project Gutenberg eBooks may be modified and printed and given away—you may do practically ANYTHING in the United States with eBooks not protected by U.S. copyright law. Redistribution is subject to the trademark license, especially commercial redistribution.

START: FULL LICENSE

THE FULL PROJECT GUTENBERG™ LICENSE

PLEASE READ THIS BEFORE YOU DISTRIBUTE OR USE THIS WORK

To protect the Project Gutenberg™ mission of promoting the free distribution of electronic works, by using or distributing this work (or any other work associated in any way with the phrase “Project Gutenberg”), you agree to comply with all the terms of the Full Project Gutenberg License available with this file or online at www.gutenberg.org/license.

Section 1. General Terms of Use and Redistributing Project Gutenberg electronic works

1.A. By reading or using any part of this Project Gutenberg electronic work, you indicate that you have read, understand, agree to and accept all the terms of this license and intellectual property (trademark/copyright) agreement. If you do not agree to abide by all the terms of this agreement, you must cease using and return or destroy all copies of Project Gutenberg electronic works in your possession. If you paid a fee for obtaining a copy of or access to a Project Gutenberg electronic work and you do not agree to be bound by the terms of this agreement, you may obtain a refund from the person or entity to whom you paid the fee as set forth in paragraph 1.E.8.

1.B. “Project Gutenberg” is a registered trademark. It may only be used on or associated in any way with an electronic work by people who agree to be bound by the terms of this agreement. There are a few things that you can do with most Project Gutenberg electronic works even without complying with the full terms of this agreement. See paragraph 1.C below. There are a lot of things you can do with Project Gutenberg electronic works if you follow the terms of this agreement and help preserve free future access to Project Gutenberg electronic works. See paragraph 1.E below.

1.C. The Project Gutenberg Literary Archive Foundation (“the Foundation” or PGLAF), owns a compilation copyright in the

collection of Project Gutenberg electronic works. Nearly all the individual works in the collection are in the public domain in the United States. If an individual work is unprotected by copyright law in the United States and you are located in the United States, we do not claim a right to prevent you from copying, distributing, performing, displaying or creating derivative works based on the work as long as all references to Project Gutenberg are removed. Of course, we hope that you will support the Project Gutenberg mission of promoting free access to electronic works by freely sharing Project Gutenberg works in compliance with the terms of this agreement for keeping the Project Gutenberg name associated with the work. You can easily comply with the terms of this agreement by keeping this work in the same format with its attached full Project Gutenberg License when you share it without charge with others.

1.D. The copyright laws of the place where you are located also govern what you can do with this work. Copyright laws in most countries are in a constant state of change. If you are outside the United States, check the laws of your country in addition to the terms of this agreement before downloading, copying, displaying, performing, distributing or creating derivative works based on this work or any other Project Gutenberg work. The Foundation makes no representations concerning the copyright status of any work in any country other than the United States.

1.E. Unless you have removed all references to Project Gutenberg:

1.E.1. The following sentence, with active links to, or other immediate access to, the full Project Gutenberg License must appear prominently whenever any copy of a Project Gutenberg work (any work on which the phrase “Project Gutenberg” appears, or with which the phrase “Project Gutenberg” is associated) is accessed, displayed, performed, viewed, copied or distributed:

This eBook is for the use of anyone anywhere in the United States and most other parts of the world at no cost and with almost no restrictions whatsoever. You may copy it, give it away or re-use it under the terms of the Project Gutenberg™ License included with

this eBook or online at www.gutenberg.org. If you are not located in the United States, you will have to check the laws of the country where you are located before using this eBook.

1.E.2. If an individual Project Gutenberg electronic work is derived from texts not protected by U.S. copyright law (does not contain a notice indicating that it is posted with permission of the copyright holder), the work can be copied and distributed to anyone in the United States without paying any fees or charges. If you are redistributing or providing access to a work with the phrase “Project Gutenberg” associated with or appearing on the work, you must comply either with the requirements of paragraphs 1.E.1 through 1.E.7 or obtain permission for the use of the work and the Project Gutenberg trademark as set forth in paragraphs 1.E.8 or 1.E.9.

1.E.3. If an individual Project Gutenberg electronic work is posted with the permission of the copyright holder, your use and distribution must comply with both paragraphs 1.E.1 through 1.E.7 and any additional terms imposed by the copyright holder. Additional terms will be linked to the Project Gutenberg License for all works posted with the permission of the copyright holder found at the beginning of this work.

1.E.4. Do not unlink or detach or remove the full Project Gutenberg License terms from this work, or any files containing a part of this work or any other work associated with Project Gutenberg.

1.E.5. Do not copy, display, perform, distribute or redistribute this electronic work, or any part of this electronic work, without prominently displaying the sentence set forth in paragraph 1.E.1 with active links or immediate access to the full terms of the Project Gutenberg License.

1.E.6. You may convert to and distribute this work in any binary, compressed, marked up, nonproprietary or proprietary form, including any word processing or hypertext form. However, if you provide access to or distribute copies of a Project Gutenberg work in a format other than “Plain Vanilla ASCII” or other format used in the official

version posted on the official Project Gutenberg website (www.gutenberg.org), you must, at no additional cost, fee or expense to the user, provide a copy, a means of exporting a copy, or a means of obtaining a copy upon request, of the work in its original “Plain Vanilla ASCII” or other form. Any alternate format must include the full Project Gutenberg License as specified in paragraph 1.E.1.

1.E.7. Do not charge a fee for access to, viewing, displaying, performing, copying or distributing any Project Gutenberg works unless you comply with paragraph 1.E.8 or 1.E.9.

1.E.8. You may charge a reasonable fee for copies of or providing access to or distributing Project Gutenberg electronic works provided that:

- You pay a royalty fee of 20% of the gross profits you derive from the use of Project Gutenberg works calculated using the method you already use to calculate your applicable taxes. The fee is owed to the owner of the Project Gutenberg trademark, but he has agreed to donate royalties under this paragraph to the Project Gutenberg Literary Archive Foundation. Royalty payments must be paid within 60 days following each date on which you prepare (or are legally required to prepare) your periodic tax returns. Royalty payments should be clearly marked as such and sent to the Project Gutenberg Literary Archive Foundation at the address specified in Section 4, “Information about donations to the Project Gutenberg Literary Archive Foundation.”
- You provide a full refund of any money paid by a user who notifies you in writing (or by e-mail) within 30 days of receipt that s/he does not agree to the terms of the full Project Gutenberg™ License. You must require such a user to return or destroy all copies of the works possessed in a physical medium and discontinue all use of and all access to other copies of Project Gutenberg™ works.
- You provide, in accordance with paragraph 1.F.3, a full refund of any money paid for a work or a replacement copy, if a defect in the electronic work is discovered and reported to you within 90 days of receipt of the work.

- You comply with all other terms of this agreement for free distribution of Project Gutenberg™ works.

1.E.9. If you wish to charge a fee or distribute a Project Gutenberg™ electronic work or group of works on different terms than are set forth in this agreement, you must obtain permission in writing from the Project Gutenberg Literary Archive Foundation, the manager of the Project Gutenberg™ trademark. Contact the Foundation as set forth in Section 3 below.

1.F.

1.F.1. Project Gutenberg volunteers and employees expend considerable effort to identify, do copyright research on, transcribe and proofread works not protected by U.S. copyright law in creating the Project Gutenberg™ collection. Despite these efforts, Project Gutenberg™ electronic works, and the medium on which they may be stored, may contain “Defects,” such as, but not limited to, incomplete, inaccurate or corrupt data, transcription errors, a copyright or other intellectual property infringement, a defective or damaged disk or other medium, a computer virus, or computer codes that damage or cannot be read by your equipment.

1.F.2. LIMITED WARRANTY, DISCLAIMER OF DAMAGES - Except for the “Right of Replacement or Refund” described in paragraph 1.F.3, the Project Gutenberg Literary Archive Foundation, the owner of the Project Gutenberg™ trademark, and any other party distributing a Project Gutenberg™ electronic work under this agreement, disclaim all liability to you for damages, costs and expenses, including legal fees. YOU AGREE THAT YOU HAVE NO REMEDIES FOR NEGLIGENCE, STRICT LIABILITY, BREACH OF WARRANTY OR BREACH OF CONTRACT EXCEPT THOSE PROVIDED IN PARAGRAPH 1.F.3. YOU AGREE THAT THE FOUNDATION, THE TRADEMARK OWNER, AND ANY DISTRIBUTOR UNDER THIS AGREEMENT WILL NOT BE LIABLE TO YOU FOR ACTUAL, DIRECT, INDIRECT, CONSEQUENTIAL, PUNITIVE OR INCIDENTAL DAMAGES

EVEN IF YOU GIVE NOTICE OF THE POSSIBILITY OF SUCH DAMAGE.

1.F.3. LIMITED RIGHT OF REPLACEMENT OR REFUND - If you discover a defect in this electronic work within 90 days of receiving it, you can receive a refund of the money (if any) you paid for it by sending a written explanation to the person you received the work from. If you received the work on a physical medium, you must return the medium with your written explanation. The person or entity that provided you with the defective work may elect to provide a replacement copy in lieu of a refund. If you received the work electronically, the person or entity providing it to you may choose to give you a second opportunity to receive the work electronically in lieu of a refund. If the second copy is also defective, you may demand a refund in writing without further opportunities to fix the problem.

1.F.4. Except for the limited right of replacement or refund set forth in paragraph 1.F.3, this work is provided to you 'AS-IS', WITH NO OTHER WARRANTIES OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY OR FITNESS FOR ANY PURPOSE.

1.F.5. Some states do not allow disclaimers of certain implied warranties or the exclusion or limitation of certain types of damages. If any disclaimer or limitation set forth in this agreement violates the law of the state applicable to this agreement, the agreement shall be interpreted to make the maximum disclaimer or limitation permitted by the applicable state law. The invalidity or unenforceability of any provision of this agreement shall not void the remaining provisions.

1.F.6. INDEMNITY - You agree to indemnify and hold the Foundation, the trademark owner, any agent or employee of the Foundation, anyone providing copies of Project Gutenberg™ electronic works in accordance with this agreement, and any volunteers associated with the production, promotion and distribution of Project Gutenberg™ electronic works, harmless from all liability, costs and expenses, including legal fees, that arise directly or indirectly from any of the following which you do or cause to occur: (a)

distribution of this or any Project Gutenberg work, (b) alteration, modification, or additions or deletions to any Project Gutenberg work, and (c) any Defect you cause.

Section 2. Information about the Mission of Project Gutenberg

Project Gutenberg is synonymous with the free distribution of electronic works in formats readable by the widest variety of computers including obsolete, old, middle-aged and new computers. It exists because of the efforts of hundreds of volunteers and donations from people in all walks of life.

Volunteers and financial support to provide volunteers with the assistance they need are critical to reaching Project Gutenberg's goals and ensuring that the Project Gutenberg collection will remain freely available for generations to come. In 2001, the Project Gutenberg Literary Archive Foundation was created to provide a secure and permanent future for Project Gutenberg and future generations. To learn more about the Project Gutenberg Literary Archive Foundation and how your efforts and donations can help, see Sections 3 and 4 and the Foundation information page at www.gutenberg.org.

Section 3. Information about the Project Gutenberg Literary Archive Foundation

The Project Gutenberg Literary Archive Foundation is a non-profit 501(c)(3) educational corporation organized under the laws of the state of Mississippi and granted tax exempt status by the Internal Revenue Service. The Foundation's EIN or federal tax identification number is 64-6221541. Contributions to the Project Gutenberg Literary Archive Foundation are tax deductible to the full extent permitted by U.S. federal laws and your state's laws.

The Foundation's business office is located at 41 Watchung Plaza #516, Montclair NJ 07042, USA, +1 (862) 621-9288. Email contact

links and up to date contact information can be found at the Foundation's website and official page at www.gutenberg.org/contact

Section 4. Information about Donations to the Project Gutenberg Literary Archive Foundation

Project Gutenberg™ depends upon and cannot survive without widespread public support and donations to carry out its mission of increasing the number of public domain and licensed works that can be freely distributed in machine-readable form accessible by the widest array of equipment including outdated equipment. Many small donations (\$1 to \$5,000) are particularly important to maintaining tax exempt status with the IRS.

The Foundation is committed to complying with the laws regulating charities and charitable donations in all 50 states of the United States. Compliance requirements are not uniform and it takes a considerable effort, much paperwork and many fees to meet and keep up with these requirements. We do not solicit donations in locations where we have not received written confirmation of compliance. To SEND DONATIONS or determine the status of compliance for any particular state visit www.gutenberg.org/donate.

While we cannot and do not solicit contributions from states where we have not met the solicitation requirements, we know of no prohibition against accepting unsolicited donations from donors in such states who approach us with offers to donate.

International donations are gratefully accepted, but we cannot make any statements concerning tax treatment of donations received from outside the United States. U.S. laws alone swamp our small staff.

Please check the Project Gutenberg web pages for current donation methods and addresses. Donations are accepted in a number of other ways including checks, online payments and credit card donations. To donate, please visit: www.gutenberg.org/donate.

Section 5. General Information About Project Gutenberg electronic works

Professor Michael S. Hart was the originator of the Project Gutenberg concept of a library of electronic works that could be freely shared with anyone. For forty years, he produced and distributed Project Gutenberg eBooks with only a loose network of volunteer support.

Project Gutenberg eBooks are often created from several printed editions, all of which are confirmed as not protected by copyright in the U.S. unless a copyright notice is included. Thus, we do not necessarily keep eBooks in compliance with any particular paper edition.

Most people start at our website which has the main PG search facility: www.gutenberg.org.

This website includes information about Project Gutenberg, including how to make donations to the Project Gutenberg Literary Archive Foundation, how to help produce our new eBooks, and how to subscribe to our email newsletter to hear about new eBooks.